PacketArch

OT Traffic Simulation Platform

Protocol-Accurate Traffic Generation for Industrial Networks

Version 1.0 | January 2026

The Challenge

Why OT Security Testing is Difficult

- **Physical Testbeds are Expensive** - Real PLCs cost thousands each
- **Production Networks are Off-Limits** - Can't test on live systems
- **Lack of Realistic Traffic** - Generic generators don't understand OT
- **Device Diversity** - Each vendor has unique fingerprints

The Solution

PacketArch: Protocol-Accurate OT Traffic Generation

PacketArch generates **realistic OT network traffic** that mirrors actual industrial devices.

Key Capabilities:

- Design network scenarios visually with drag-and-drop
- Generate traffic for 6+ industrial protocols
- Learn device fingerprints from real PCAP captures
- Deploy traffic agents to any network location

Value Propositions

Capability	Benefit
Visual Scenario Studio	Design OT networks in minutes
AI-Powered Generation	Describe scenarios in natural language
Multi-Protocol Support	Modbus, EtherNet/IP, PROFINET, S7, BACnet, SNMP
Live Traffic Injection	Deploy agents to inject into real networks
PCAP Learning	Create templates from captured traffic

Platform Overview

High-Level Architecture

```
┌──────────────────────────────────────────────────┐
│               PacketArch Server                   │
│  Frontend ◄──► Backend ◄──► Database             │
│  (React)       (FastAPI)     (PostgreSQL)        │
└──────────────────────┬───────────────────────────┘
                       │ WebSocket
        ┌──────────────┼──────────────┐
        ▼              ▼              ▼
  [ Agent 1 ]    [ Agent 2 ]    [ Agent N ]
  Lab Network    DMZ Network    Production
```

Technology Stack

Layer	Technologies
Frontend	React 18, Vite, TypeScript, Ant Design, @xyflow/react
Backend	FastAPI, SQLAlchemy 2.0, Scapy, Celery
Database	PostgreSQL, Redis
Deployment	Docker Compose, nginx

Supported Protocols

Protocol	Port(s)	Status
Modbus TCP	502	Production
EtherNet/IP	44818, 2222	Production
PROFINET	Layer 2	Production
S7comm	102	Production
BACnet/IP	47808	Production
SNMP/NTCIP	161, 162	Production
OPC UA, DNP3, IEC 104	Various	Planned

Core Features

Visual Scenario Studio

Canvas-Based Network Design

- **Device Palette** - PLCs, HMIs, RTUs, switches, sensors
- **Zone Containers** - Group devices by network segment
- **Protocol Flows** - Visual connections showing communication
- **Auto IP Assignment** - Each scenario gets unique `/16` range
- **Undo/Redo** - Full history tracking

Device Templates

Unified Fingerprint System

Source	Description
VENDOR_BUILTIN	Pre-packaged fingerprints (Siemens, Rockwell, etc.)
PCAP_LEARNED	Templates extracted from captured traffic
USER_CREATED	Custom templates created by users

Templates include: Network signatures, Protocol identities, Timing models

Traffic Generation

Protocol Engines with State Machines

Each protocol implements a **stateful engine**:

- `generate_startup_sequence()` - Protocol initialization
- `generate_poll_cycle()` - Request/response exchange
- `generate_shutdown_sequence()` - Clean disconnect

Output: PCAP files or live traffic via agents

Industry Templates

Vertical	Protocols
Manufacturing	PROFINET, EtherNet/IP
Water/Wastewater	Modbus, DNP3
Energy/Power	IEC 104, Modbus
Oil & Gas	Modbus, OPC UA
Building Automation	BACnet, Modbus
Transportation/ITS	SNMP/NTCIP

PCAP Learning

Learn from Real Traffic

1. **Upload** - PCAP file to PacketArch
2. **Analyze** - Extract protocols and patterns
3. **Extract** - Device fingerprints and timing
4. **Create** - Templates for traffic generation

Anomaly Injection

Security Testing Capabilities

Category	Examples
Protocol Violations	Invalid function codes, malformed packets
Timing Anomalies	Burst traffic, unusual poll intervals
Address Anomalies	Out-of-range registers, spoofed IPs
Behavioral Anomalies	Unexpected commands, sequence breaks

AI Integration

Natural Language Scenario Generation

> "Create a manufacturing cell with a Siemens S7-1500 PLC controlling two VFDs over PROFINET"

- **Scenario Generation** from descriptions
- **Device Suggestions** for use cases
- **Protocol Configuration** assistance
- **Troubleshooting** help

Cisco Cyber Vision Integration

Feature	Description
Device Discovery	Fetch devices from Cyber Vision
Scenario Comparison	Match PacketArch vs CV inventory
Device Matching	MAC (100%) or IP (95%) confidence
Enrichment Push	Send vendor/model/firmware to CV

Remote Traffic Agents

Agent Architecture

WebSocket "Phone Home" Model

```
         PacketArch Server
                │
                │ WebSocket (wss://)
      ┌─────────┼─────────┐
      ▼         ▼         ▼
  Agent 1   Agent 2   Agent 3
```

- Agents **initiate** outbound connections
- **No inbound ports** required on agent hosts
- Works through NAT and firewalls

Agent Benefits

- **No inbound ports** - Outbound connections only
- **Firewall friendly** - Works through NAT
- **Instant commands** - Server pushes via WebSocket
- **Central management** - Control all from single UI
- **Auto-reconnect** - Handles network issues
- **Central updates** - Push new versions remotely

Agent Installation

One-Command Deployment

```
curl -fsSL https://server/agent/install.sh | sudo bash -s -- \
  --server https://10.10.20.231 \
  --name "Lab-Agent-01" \
  --register
```

Deploys Docker container with auto-restart and Watchtower.

WebSocket Protocol

Server → Agent:

`START_SCENARIO`, `STOP_SCENARIO`, `UPDATE_AGENT`, `PING`

Agent → Server:

`HEARTBEAT`, `STATUS`, `INTERFACES`, `ERROR`, `UPDATE_STATUS`

Technical Deep Dive

Protocol Engine Architecture

```python
class ProtocolEngine(ABC):
    @abstractmethod
    def generate_startup_sequence(self) -> list[Packet]:
        pass

    @abstractmethod
    def generate_poll_cycle(self) -> list[Packet]:
        pass

    @abstractmethod
    def generate_shutdown_sequence(self) -> list[Packet]:
        pass
```

Identity System

MAC Address Generation

```
mac = generate_mac(vendor="Siemens", device_type="PLC")
# Returns: "00:1C:06:XX:XX:XX" (Siemens OUI)

mac = generate_mac(vendor="Rockwell", device_type="PLC")
# Returns: "00:00:BC:XX:XX:XX" (Rockwell OUI)
```

Protocol-specific identity builders for Modbus, EtherNet/IP, PROFINET, S7, BACnet, SNMP.

Timing System

Statistical Response Delay Simulation

Distribution	Use Case
Gaussian	Normal device timing
Lognormal	Network delays (skewed)
Uniform	Even distribution
Learned	Replay from captured samples

API Architecture

Category	Base Path
Scenarios	`/api/v1/scenarios`
Templates	`/api/v1/templates`
Learning	`/api/v1/learning`
Agents	`/api/v1/agents`
Deployments	`/api/v1/deployments`
Cyber Vision	`/api/v1/cyber-vision`

WebSocket: `/ws/agent?token=xxx`

Security Model

Data	Protection
Agent Tokens	Bcrypt hashed
API Keys	Fernet encryption
TLS Certificates	Encrypted storage
WebSocket	TLS (wss://)

Use Cases

Security Sensor Validation

Challenge: Security sensors need traffic to detect threats

Solution: Generate known-good and anomalous traffic

- **Baseline** - Normal traffic, verify no false positives
- **Reconnaissance** - Scanning patterns
- **Protocol Violations** - Malformed packets
- **Unauthorized Access** - Write to read-only devices

Red Team / Blue Team

Exercise	PacketArch Role
Attack Simulation	Generate recon and exploitation traffic
Detection Tuning	Known-bad traffic for rule development
Incident Response	Realistic attack scenarios for practice

Network Simulation

Scenario	Description
Lab Realism	Background traffic for production feel
Integration Testing	Test devices against simulated network
Training	Realistic OT networks for education
Demos	Showcase products with realistic traffic

Deployment & Operations

Development Setup

```
# Clone and start
git clone git@github.com:ip-aegis/PacketArch.git
cd docker && docker-compose -f docker-compose.dev.yml up -d
cd ../backend && poetry install
cd ../frontend && pnpm install

# Run
backend:  poetry run uvicorn app.main:app --reload --port 8001
frontend: pnpm dev
```

Production Deployment

```
# Deploy all services
docker compose up -d --build

# Rebuild specific service
docker compose up -d --build backend

# View logs
docker compose logs -f backend
```

Access via HTTPS on port 443 (nginx reverse proxy with TLS).

Roadmap & Resources

Planned Features

Upcoming Protocols:

- OPC UA (Q2 2026)
- DNP3 (Q2 2026)
- IEC 104 (Q3 2026)

Platform Enhancements:

- Traffic Replay
- Scenario Scheduling
- Multi-Agent Coordination
- Plugin System

Resources

Repository: `github.com/ip-aegis/PacketArch`

Documentation:

- `CLAUDE.md` - Developer guide
- `/api/docs` - Swagger UI
- `docs/ADDING_NEW_PROTOCOLS.md`

Questions?

PacketArch - OT Traffic Simulation Platform

GitHub: `github.com/ip-aegis/PacketArch`

Thank You

PacketArch

Protocol-Accurate OT Traffic Generation

Securing industrial networks through realistic simulation

January 2026